

K-Nearest Neighbors (KNN) Algorithm

A Complete Academic Reference Guide

Machine Learning | Supervised Learning | Classification & Regression

Exam-Ready Notes

Python Code Included

Manual Examples

Interview Q&A

Topics Covered: Definition • Math Foundations • Distance Metrics

Algorithm Steps • Python Code • Complexity • Applications

Interview Questions • Comparisons • Advantages & Disadvantages



Machine Learning | Comprehensive Study Material | Academic Edition
Suitable for University Exams, Assignments & Technical Interviews

TABLE OF CONTENTS

1. Introduction to Machine Learning & KNN Context
2. Definition of KNN
3. Intuition Behind KNN
4. Mathematical Foundation of KNN
5. Visual Explanation & Diagrams
6. KNN Algorithm — Step-by-Step
7. Pseudocode of KNN Algorithm
8. Numerical Example — Manual Calculation
9. KNN from Scratch in Python
10. KNN Implementation using Scikit-Learn
11. Flowchart of KNN Working Process
12. Choosing the Best K Value
13. Advantages of KNN
14. Disadvantages of KNN
15. Applications of KNN
16. Real-World Examples
17. Comparison with Other Algorithms
18. Important Properties of KNN
19. Time & Space Complexity
20. Interview Questions & Answers
21. Short Revision Notes
22. Summary — KNN in 5 Key Points

1 INTRODUCTION TO MACHINE LEARNING & KNN CONTEXT

1.1 What is Machine Learning?

Machine Learning (ML) is a branch of Artificial Intelligence (AI) that enables computers to **learn from data** and improve their performance on tasks without being explicitly programmed for each scenario. Instead of writing hard rules, we feed data to an algorithm and let it discover patterns automatically.

Three main categories of Machine Learning:

Type	Description	Examples
Supervised Learning	Trains on labelled data (input→output pairs); learns a mapping function.	KNN, SVM, Decision Tree, Linear Regression
Unsupervised Learning	Finds hidden patterns in unlabelled data.	K-Means, PCA, DBSCAN
Reinforcement Learning	Agent learns by interacting with environment and receiving rewards/penalties.	AlphaGo, Game bots, Robotics

1.2 Where Does KNN Fit in Machine Learning?

KNN (K-Nearest Neighbors) sits firmly within **Supervised Learning**. It can be used for both **Classification** (predicting a category) and **Regression** (predicting a continuous value). It is one of the simplest yet most powerful algorithms — making it an ideal starting point for learning ML.

KNN in the ML Landscape:

- **Learning type:** Supervised Learning
- **Task type:** Classification AND Regression
- **Model type:** Non-parametric, Instance-based (Lazy Learner)
- **Learning style:** Memorise training data; decide at prediction time
- **Complexity:** Conceptually simple; computationally expensive at scale

2 DEFINITION OF KNN

2.1 Simple Definition

KNN is a supervised machine learning algorithm that classifies a new data point based on the majority class of its K closest neighbours in the training dataset. It uses a distance metric (e.g., Euclidean) to measure closeness.

2.2 Everyday Analogy

Imagine you move to a new neighbourhood and want to know what kind of area it is. You ask your **K nearest neighbours** about their lifestyle. If most of them are professionals, you conclude "this is a professional neighbourhood." KNN works exactly the same way — it checks what class the nearest training examples belong to and votes.

2.3 Technical Definition

Given a training dataset $D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$ where $x_i \in \mathbb{R}^d$ is a feature vector and y_i is the class label, KNN predicts the label of a new query point x_q by:

1. Computing distance $d(x_q, x_i)$ for all training points.
2. Sorting distances in ascending order.
3. Selecting the K training points with the smallest distances (K-Nearest Neighbors).
4. For **Classification**: Returning the most frequent class label among the K neighbours (majority voting).
5. For **Regression**: Returning the mean (or weighted mean) of the K neighbours' output values.

3 INTUITION BEHIND KNN

3.1 The Idea of Similarity

The fundamental assumption of KNN is: **"Similar inputs produce similar outputs."** Data points that are close to each other in feature space tend to belong to the same class. This is sometimes called the *continuity assumption*.

3.2 Concept of Neighbours

A **neighbour** is a training data point that is close to the query point according to some distance metric. K controls how many neighbours we consult. A small K means very local decisions; a large K means more global decisions.

3.3 Why Distance Matters

Distance is the core mechanism that quantifies similarity. Two common intuitions:

- **Closer = More Similar:** A point very near the query is likely from the same class.
- **Farther = Less Relevant:** Points far away should influence the prediction less.

In weighted KNN, closer neighbours are assigned higher weights so they influence the prediction more strongly than distant ones.

KEY INSIGHT: KNN makes NO assumptions about the shape of the decision boundary. It learns any arbitrary boundary shape — making it very flexible. The decision boundary adapts directly to the training data distribution.

4 MATHEMATICAL FOUNDATION OF KNN

4.1 Distance Metric Concept

A distance metric $d(p, q)$ measures how far apart two points p and q are in feature space. For KNN to work correctly, the chosen metric must satisfy:

- Non-negativity: $d(p, q) \geq 0$
- Identity: $d(p, q) = 0$ if and only if $p = q$
- Symmetry: $d(p, q) = d(q, p)$
- Triangle Inequality: $d(p, r) \leq d(p, q) + d(q, r)$

4.2 Euclidean Distance (Most Common)

Euclidean distance is the straight-line (as-the-crow-flies) distance between two points in n -dimensional space.

$$d_E(p, q) = \sqrt{(p_1 - q_1)^2 + (p_2 - q_2)^2 + \dots + (p_n - q_n)^2}$$

$$d_E(p, q) = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$$

Symbol	Meaning
$p = (p_1, p_2, \dots, p_n)$	First data point with n features
$q = (q_1, q_2, \dots, q_n)$	Second data point (query point) with n features
$p_i - q_i$	Difference in the i -th feature between the two points
$(p_i - q_i)^2$	Squared difference (always positive)
Σ (sum)	Sum over all n features (dimensions)
$\sqrt{}$ (square root)	Converts squared sum back to original units
n	Number of features (dimensions) in the dataset

4.3 Manhattan Distance (City Block / L1 Norm)

Manhattan distance measures the total absolute difference along each axis — like navigating city blocks on a grid.

$$d_M(p, q) = \sum_{i=1}^n |p_i - q_i|$$

- No squaring — uses absolute values instead.
- More robust to outliers than Euclidean distance.
- Preferred when features are measured on different scales or for high-dimensional data.

4.4 Minkowski Distance (Generalised Form)

Minkowski distance is the generalisation that includes both Euclidean and Manhattan as special cases:

$$d_{\text{Mink}}(p, q) = \left(\sum_{i=1}^n |p_i - q_i|^r \right)^{1/r}$$

Parameter r	Resulting Distance Metric
$r = 1$	Manhattan Distance (L1 Norm)
$r = 2$	Euclidean Distance (L2 Norm) ← most common for KNN
$r = \infty$	Chebyshev Distance (maximum absolute difference)
$r \rightarrow 0$	Hamming Distance approximation

4.5 Other Distance Metrics

Metric	Formula / Description	Use Case
Hamming	Count of positions where symbols differ	Categorical / text data
Cosine	$1 - \cos(\text{angle between vectors})$	Text, NLP, documents
Chebyshev	$\max p_i - q_i $ over all i	Chess king moves, warehouse routing
Mahalanobis	Accounts for feature correlations and scale	Correlated features

5 VISUAL EXPLANATION & DIAGRAMS

5.1 Scatter Plot — KNN Classification with K=3

The diagram below shows two classes of data points (Class A = blue circles, Class B = orange squares). The green point is the unknown query point. The red dashed circle represents the search radius enclosing K=3 nearest neighbours.

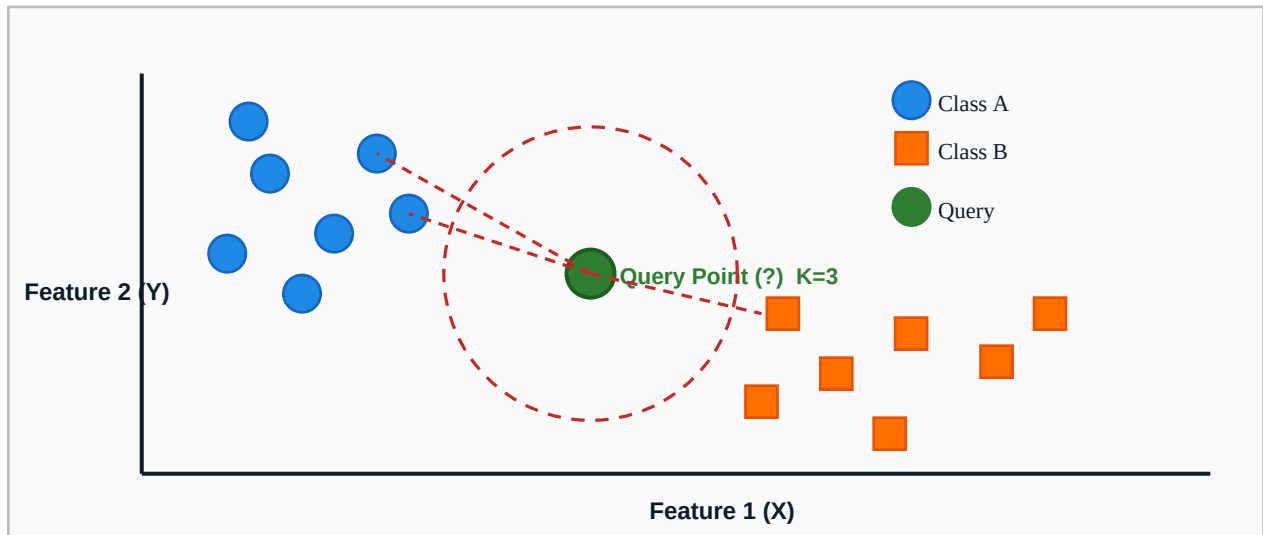


Figure 5.1 — KNN Classification: Query point (green) identifies its 3 nearest neighbours. 2 of 3 are Class A (blue) → Predicted Class = A

5.2 KNN Decision Boundary Visualisation

The decision boundary in KNN is the line (or surface) that separates the two class regions. Unlike linear classifiers, KNN's boundary is **non-linear and adaptive** — it bends around the training data. As K increases, the boundary becomes smoother.

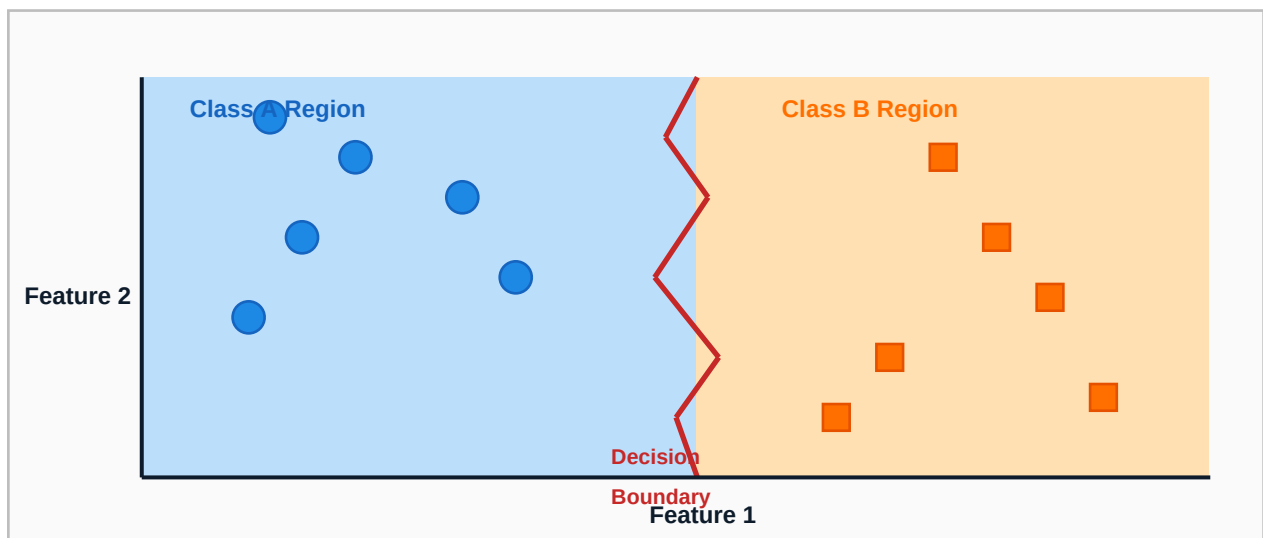


Figure 5.2 — KNN Decision Boundary: Blue region = Class A territory; Orange region = Class B territory. Red jagged line = non-linear decision boundary.

6 KNN ALGORITHM — STEP-BY-STEP

The KNN algorithm is remarkably straightforward. Below is the complete process:

Step 1	Choose the Value of K Select a positive integer K. This hyperparameter determines how many neighbours will vote on the prediction. Common choice: $K = \sqrt{n}$ (where n = number of training samples). Always use an odd K for binary classification to avoid ties.
Step 2	Calculate Distance Compute the distance between the query point x_q and every single training data point x_i using a chosen distance metric (usually Euclidean). Store each (distance, class_label) pair.
Step 3	Sort Distances Sort all computed distances in ascending order (smallest distance first). This ranks training points from most similar to least similar relative to the query.
Step 4	Select K Nearest Neighbours Take the top-K entries from the sorted list. These K training points are the K-Nearest Neighbours of the query point.
Step 5	Perform Majority Voting (Classification) Count the occurrences of each class label among the K neighbours. The class with the highest count wins. For regression: compute the mean of K neighbours' values.
Step 6	Assign Class Label / Output Value Assign the winning class label (majority vote) to the query point. Output this as the predicted class. The algorithm is now complete.

7 PSEUDOCODE OF KNN ALGORITHM

The following pseudocode is suitable for exam and assignment answers:

```
ALGORITHM: KNN_Classifier(TrainingSet D, QueryPoint x_q, Integer K)
```

```
INPUT:
```

```
D = { (x1,y1), (x2,y2), ..., (xn,yn) } – training data
```

```
x_q = Query point to classify
```

```
K = Number of nearest neighbours
```

```
OUTPUT:
```

```
y_pred = Predicted class label for x_q
```

```
BEGIN
```

```
distances ← empty list
```

```
FOR each training sample (xi, yi) IN D DO
```

```
dist ← EuclideanDistance(x_q, xi)
```

```
= sqrt( sum( (x_q[j] - xi[j])^2 ) for j = 1 to d )
```

```
APPEND (dist, yi) TO distances
```

```
END FOR
```

```
sorted_distances ← SORT distances by dist (ASCENDING)
```

```
k_neighbours ← sorted_distances[0 : K] // first K elements
```

```
class_votes ← COUNT occurrences of each class in k_neighbours
```

```
y_pred ← class with MAXIMUM votes in class_votes
```

```
RETURN y_pred
```

```
END
```

```
// For REGRESSION, replace majority vote with:
```

```
// y_pred = MEAN( yi for (dist, yi) in k_neighbours )
```

7.1 Euclidean Distance Function (Pseudocode)

```
FUNCTION EuclideanDistance(p, q):
```

```
total ← 0
```

```
FOR i FROM 1 TO d DO
```

```
total ← total + (p[i] - q[i])^2
```

```
END FOR
```

```
RETURN sqrt(total)
```

```
END FUNCTION
```

8 NUMERICAL EXAMPLE — MANUAL CALCULATION

Let us manually run KNN on a small dataset. This is commonly asked in exams!

8.1 Training Dataset

Point ID	Feature X ¹ (Hours Studied)	Feature X ² (Prev Score)	Class Label
P1	1	2	Fail
P2	2	3	Fail
P3	3	6	Pass
P4	5	7	Pass
P5	6	8	Pass
P6	7	9	Pass
Query (?)	4	5	???

8.2 Step-by-Step Distance Calculation (K = 3)

Query Point Q = (4, 5). Using **Euclidean Distance**:

Point	X ¹	X ²	Class	Distance Formula	Simplified	Exact	Distance
P1	1	2	Fail	$\sqrt{[(4-1)^2+(5-2)^2]}$	$\sqrt{[9+9]}$	$\sqrt{18}$	4.243
P2	2	3	Fail	$\sqrt{[(4-2)^2+(5-3)^2]}$	$\sqrt{[4+4]}$	$\sqrt{8}$	2.828
P3	3	6	Pass	$\sqrt{[(4-3)^2+(5-6)^2]}$	$\sqrt{[1+1]}$	$\sqrt{2}$	1.414
P4	5	7	Pass	$\sqrt{[(4-5)^2+(5-7)^2]}$	$\sqrt{[1+4]}$	$\sqrt{5}$	2.236
P5	6	8	Pass	$\sqrt{[(4-6)^2+(5-8)^2]}$	$\sqrt{[4+9]}$	$\sqrt{13}$	3.606
P6	7	9	Pass	$\sqrt{[(4-7)^2+(5-9)^2]}$	$\sqrt{[9+16]}$	$\sqrt{25}$	5.000

8.3 Sorted Distances (Ascending Order)

Rank	Point	Distance	Class Label
1st (nearest)	P3	1.414	Pass ✓
2nd	P4	2.236	Pass ✓
3rd	P2	2.828	Fail ✗
4th	P5	3.606	Pass
5th	P1	4.243	Fail
6th (farthest)	P6	5.000	Pass

8.4 Majority Voting & Final Prediction

$K = 3 \rightarrow$ Top 3 Neighbours: P3 (Pass), P4 (Pass), P2 (Fail)

Vote Count: Pass = 2 | Fail = 1

MAJORITY CLASS = Pass (2 out of 3 votes)

FINAL PREDICTION: Query Point $Q = (4, 5) \rightarrow$ Predicted Class = PASS

9 KNN FROM SCRATCH IN PYTHON

Below is a complete, well-commented Python implementation of KNN without any ML library. Every line is explained.

```
# ■■■ KNN FROM SCRATCH ■■■■
```

```
# Import only standard math library – no sklearn needed
```

```
import math
```

```
from collections import Counter
```

```
# ■■■ STEP 1: DATASET ■■■■
```

```
# Each row: [Feature_X1, Feature_X2, Class_Label]
```

```
training_data = [  
[1, 2, 'Fail'], # Point P1  
[2, 3, 'Fail'], # Point P2  
[3, 6, 'Pass'], # Point P3  
[5, 7, 'Pass'], # Point P4  
[6, 8, 'Pass'], # Point P5  
[7, 9, 'Pass'], # Point P6  
  
]  

```

```
# ■■■ STEP 2: DISTANCE FUNCTION ■■■■
```

```
def euclidean_distance(point1, point2):  
'''  
Calculates Euclidean distance between two n-dimensional points.  
point1, point2: lists of numeric feature values (excluding label)  
'''  
total = 0  
for i in range(len(point1)): # Loop through each feature  
diff = point1[i] - point2[i] # Feature difference  
total += diff ** 2 # Square and accumulate  
return math.sqrt(total) # Square root of sum  

```

```
# ■■■ STEP 3: KNN FUNCTION ■■■■
```

```
def knn_predict(training_data, query_point, K):  
'''  
Predicts the class of a query_point using KNN.  
training_data : list of [features..., label]  
query_point : list of feature values [x1, x2, ...]  
K : number of nearest neighbours to consider  
'''  
distances = []  
  
# Calculate distance from query to every training point  
for data_point in training_data:  
features = data_point[:-1] # All columns except last (label)  
label = data_point[-1] # Last column is the class label  
dist = euclidean_distance(query_point, features)  
distances.append((dist, label)) # Store (distance, class) pair  
  
# Sort by distance – ascending (closest first)
```


10 KNN IMPLEMENTATION USING SCIKIT-LEARN

Scikit-Learn's `KNeighborsClassifier` makes production-ready KNN implementation straightforward. Below is the complete code with line-by-line explanation.

```
# ■■■ KNN WITH SCIKIT-LEARN  
# Import KNN classifier from sklearn's neighbors module  
from sklearn.neighbors import KNeighborsClassifier  
  
# Import train_test_split to divide data into training and testing sets  
from sklearn.model_selection import train_test_split  
  
# Import accuracy_score to evaluate model performance  
from sklearn.metrics import accuracy_score, classification_report  
  
# Import numpy for numerical array operations  
import numpy as np  
  
# ■■ DATASET  
# Feature matrix X: rows = samples, columns = features  
X = np.array([[1,2],[2,3],[3,6],[5,7],[6,8],[7,9],[4,4],[3,5]])  
# Target labels y: corresponding class for each sample  
y = np.array(['Fail','Fail','Pass','Pass','Pass','Pass','Fail','Pass'])  
  
# ■■ TRAIN-TEST SPLIT  
# Split 75% training, 25% testing; random_state for reproducibility  
X_train, X_test, y_train, y_test = train_test_split(  
X, y, test_size=0.25, random_state=42  
)  
  
# ■■ CREATE KNN MODEL  
# n_neighbors=3 means we consider 3 nearest neighbours  
# metric='euclidean' specifies the distance formula to use  
# weights='uniform' means all neighbours vote equally  
knn_model = KNeighborsClassifier(  
n_neighbors=3,  
metric='euclidean',  
weights='uniform'  
)  
  
# ■■ TRAIN THE MODEL  
# .fit() stores training data – KNN is a lazy learner (no real training)  
knn_model.fit(X_train, y_train)  
  
# ■■ MAKE PREDICTIONS  
# .predict() classifies each test sample  
y_predicted = knn_model.predict(X_test)
```


11 FLOWCHART OF KNN WORKING PROCESS

The flowchart below illustrates the complete decision flow of the KNN algorithm from receiving input to producing the final prediction.



12 CHOOSING THE BEST VALUE OF K

12.1 Why K Matters

The value of K is the most critical hyperparameter in KNN. It controls the **bias-variance trade-off** of the model. There is no universally perfect K — it must be tuned for each problem.

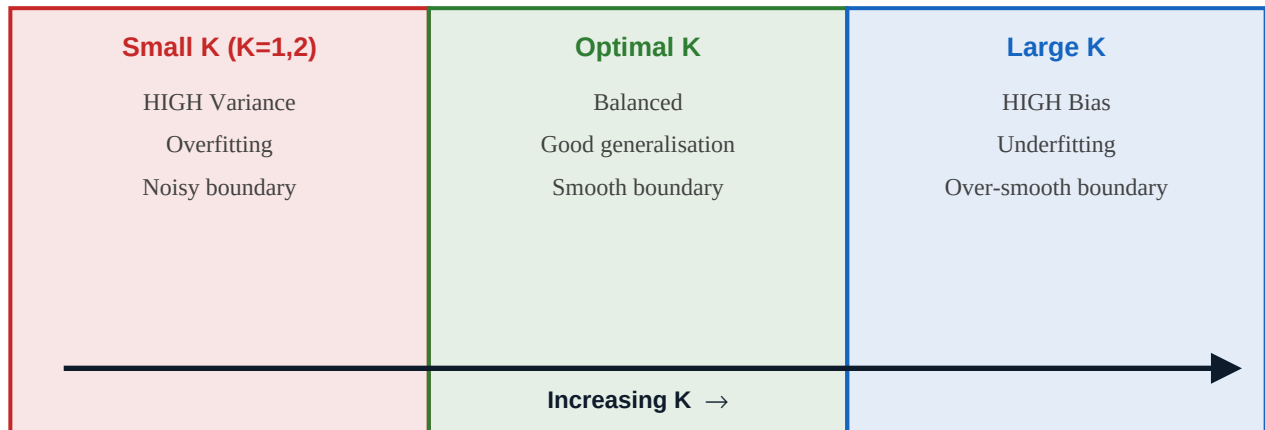


Figure 12.1 — Effect of K on model behaviour. Left=overfitting (K too small), Centre=optimal K, Right=underfitting (K too large).

12.2 Effect of Small K

- Model is highly sensitive to individual training points.
- The decision boundary is very jagged and irregular.
- Extremely susceptible to noise and outliers in training data.
- **Overfitting** — excellent on training data, poor on unseen data.
- K=1 is the extreme case: prediction = label of single nearest neighbour.

12.3 Effect of Large K

- Model considers a very large neighbourhood — becomes less local.
- Decision boundary becomes overly smooth and may miss fine structure.
- **Underfitting** — may misclassify points whose class is a local minority.
- Computationally more expensive at prediction time.
- K=N (all training points) is degenerate — always predicts the majority class.

12.4 Rules of Thumb for Selecting K

Method	Description
Square Root Rule	$K \approx \sqrt{n}$ where n = total number of training samples. Good default starting point.
Always Use Odd K	For binary classification, use odd K to avoid ties in voting.
Cross-Validation	Use k-fold cross-validation to test multiple K values and pick best accuracy.
Elbow Method	Plot validation error vs K; choose K at the 'elbow' where error stabilises.
Domain Knowledge	Small K for complex non-linear boundaries; larger K for smooth boundaries.

13 ADVANTAGES OF KNN

- ✓ **Simple & Intuitive:** KNN requires no training phase and is extremely easy to understand and implement. The concept mirrors human reasoning ('tell me who your neighbours are').
- ✓ **No Training Phase:** KNN is a lazy learner — it simply memorises the training dataset. New training data can be added instantly without retraining the entire model.
- ✓ **Non-Parametric:** No assumptions about data distribution (unlike Gaussian Naive Bayes or Logistic Regression). Works well with real-world data that may not follow any neat distribution.
- ✓ **Naturally Multi-Class:** KNN handles multi-class classification directly through majority voting — no special modification needed, unlike SVM or Logistic Regression.
- ✓ **Adaptive Decision Boundary:** The decision boundary adapts to any arbitrary shape. Complex class boundaries that no linear model could capture are handled naturally.
- ✓ **Works Well on Small Datasets:** With small, well-structured datasets, KNN often achieves surprisingly high accuracy because it uses all available information at prediction time.
- ✓ **No Model Assumptions:** Does not assume linearity, independence of features, or any probability distribution — highly flexible.
- ✓ **Interpretable:** The reasoning is fully transparent: 'I classified this point as X because its 3 nearest neighbours were of class X.' Easy to explain to non-technical stakeholders.

14 DISADVANTAGES OF KNN

- ✗ **High Computational Cost at Prediction:** KNN calculates distance to every training point for each new query. With n training samples and d features, prediction time is $O(n \cdot d)$. Very slow for large datasets.
- ✗ **High Memory Usage:** The entire training dataset must be stored in memory at all times. No compression or abstraction occurs — memory = $O(n \cdot d)$.
- ✗ **Sensitive to Irrelevant Features:** If irrelevant features are included, they can distort distances and degrade performance significantly. Feature selection/engineering is critical.
- ✗ **Sensitive to Feature Scale:** Features on large numeric scales dominate the distance calculation. **Feature normalisation (Min-Max or Z-score standardisation) is mandatory.**
- ✗ **Curse of Dimensionality:** In very high-dimensional spaces, all points become approximately equidistant — the concept of 'nearest neighbour' loses meaning. PCA or dimensionality reduction is required.
- ✗ **Sensitive to Imbalanced Data:** If one class has far more samples than another, the majority class will dominate voting even for minority-class query points.
- ✗ **Choosing K is Non-Trivial:** There is no analytical formula for the best K . It requires cross-validation, domain knowledge, or heuristics — adding complexity to the workflow.
- ✗ **No Explicit Model:** Unlike Decision Trees or Linear Regression, KNN produces no learnable model parameters. It cannot be serialised as a compact formula.

15 APPLICATIONS OF KNN

#	Application Domain	How KNN is Used
1	Recommendation Systems	Netflix/Amazon find users similar to you (K nearest user profiles) and recommend what they liked
2	Image Recognition	Classify images by comparing feature vectors (pixel intensities, HOG descriptors) to labelled train
3	Medical Diagnosis	Diagnose diseases (cancer, diabetes) by comparing patient test results to K most similar diagnos
4	Fraud Detection	Flag transactions as fraudulent if their K nearest historical transactions were fraudulent.
5	Pattern Recognition	Handwritten digit recognition (MNIST) — classify digit based on nearest known digits.
6	Document Classification	Classify text documents by comparing TF-IDF feature vectors to K nearest labelled documents.
7	Credit Scoring	Banks assess loan risk by comparing applicant profiles to K most similar historical applicants.
8	Anomaly Detection	An outlier is a point whose K nearest neighbours are far away — indicates unusual/abnormal beh
9	Genetics & Bioinformatics	Find genes with similar expression profiles to understand gene function or disease mechanisms.
10	Natural Language Processing	Semantic similarity — find K nearest sentences/words in embedding space for text classification.
11	Retail & Inventory	Forecast demand for products with similar sales history profiles.
12	Computer Vision	Face recognition systems compare face embeddings to find K most similar known faces.

16 REAL-WORLD EXAMPLES

■ Spotify Music Recommendation

Spotify represents each song as a feature vector (tempo, energy, danceability, valence, acousticness). When you like a song, KNN finds the K most similar songs and adds them to your Discover Weekly playlist.

■ KNN in Healthcare — Tumour Classification

Given features of a tumour biopsy (size, shape, texture, margin type), KNN compares it to K nearest cases in medical records. If 4 of 5 nearest cases were malignant, the tumour is predicted as malignant.

■ E-Commerce Product Tagging

New products uploaded without manual tags are automatically tagged by finding K most similar products (by description embeddings) and inheriting their tags.

■ Stock Market — Similar Day Pattern

On a given trading day with certain macroeconomic indicators, KNN finds K historically similar trading days and predicts whether the market will rise or fall.

17 COMPARISON WITH OTHER ALGORITHMS

Criterion	KNN	Logistic Regression	Decision Tree	SVM
Type	Instance-based	Parametric model	Rule-based	Margin-based
Learning	Lazy (no training)	Eager (trains model)	Eager	Eager
Training time	$O(1)$ — none	$O(n \cdot d \cdot \text{iterations})$	$O(n \cdot d \cdot \log n)$	$O(n^2 \cdot d)$ to $O(n^3 \cdot d)$
Prediction time	$O(n \cdot d)$ — slow	$O(d)$ — very fast	$O(\log n)$ — fast	$O(n_{sv} \cdot d)$
Memory	Stores all data	Stores weights only	Compact tree	Support vectors only
Interpretability	High (show neighbors)	Medium (coefficients)	Very high (rules)	Low (kernel space)
Handles non-linear	Yes, naturally	No (needs poly features)	Yes	Yes (kernel trick)
Feature scaling	Required	Recommended	Not required	Required
Outlier sensitivity	High	Moderate	Low	Low
Multi-class	Yes, natural	Needs modification	Yes, natural	Needs OvR/OvO
Best use case	Small datasets, non-linear problems	Binary classification, linearly separable	Categorical features, explainable models	High-dim data, margin maximisation

18 IMPORTANT PROPERTIES OF KNN

Lazy Learner

KNN does NOT build an explicit model during training. It simply memorises the entire training dataset. All computation is deferred to prediction time. This contrasts with 'eager learners' like Decision Trees or Neural Networks that build a generalised model during training. Trade-off: Zero training time, but slow prediction.

Non-Parametric Model

KNN makes NO assumptions about the underlying probability distribution of the data. There are no parameters (weights, coefficients) to estimate. This makes KNN very flexible and applicable to data of any distribution shape. Compare: Logistic Regression assumes a linear log-odds relationship; Gaussian Naive Bayes assumes Gaussian distributions.

Instance-Based Learning

KNN belongs to the family of instance-based (or memory-based) learning algorithms. Generalisation is local: the model only generalises when a query is made, based on the instances in the immediate neighbourhood of the query. This enables KNN to represent very complex non-linear functions with no assumption about global structure.

Transductive Learning

KNN can be viewed as a transductive method: it does not learn a global function but instead makes predictions point-by-point based on local evidence. There is no model file to save — the model IS the data.

19 TIME & SPACE COMPLEXITY OF KNN

Phase	Complexity	Variables	Explanation
Training Time	$O(1)$	—	KNN does nothing during training — just stores data. True $O(1)$.
Prediction Time (Brute Force)	$O(n \cdot d)$	n = training samples d = number of features	Must compute distance to ALL n training points, each taking $O(d)$.
Prediction Time (KD-Tree)	$O(d \cdot \log n)$	n = training samples d = features	KD-Tree spatial indexing speeds up nearest neighbour search.
Prediction Time (Ball-Tree)	$O(d \cdot \log n)$	n, d as above	Ball-Tree works better in higher dimensions than KD-Tree.
Space Complexity	$O(n \cdot d)$	n = samples d = features	Must store every training point. No compression. Memory grows with $n \cdot d$.
KD-Tree Build Time	$O(n \cdot d \cdot \log n)$	n, d as above	One-time cost to build spatial index; amortised over many predictions.

19.1 Scalability Strategies

- **KD-Tree:** Divides feature space into a binary tree. Reduces prediction to $O(d \cdot \log n)$ for low d . Default in sklearn for $d < 20$.
- **Ball-Tree:** Builds nested hypersphere structure. Better for high dimensions. Use when $d > 20$.
- **Approximate Nearest Neighbour (ANN):** Libraries like FAISS, Annoy, ScaNN. Trade small accuracy loss for massive speed gain.
- **Dimensionality Reduction:** Apply PCA or t-SNE before KNN to reduce $d \rightarrow$ fewer distance calculations.
- **Data Pruning:** CNN (Condensed Nearest Neighbours) removes redundant training points while preserving decision boundary.
- **Parallelisation:** Distance calculations are embarrassingly parallel — use sklearn `n_jobs=-1` to use all CPU cores.

20 INTERVIEW QUESTIONS & ANSWERS

Q1. What is KNN and how does it work?

Answer: KNN (K-Nearest Neighbors) is a supervised, non-parametric, lazy learning algorithm. It classifies a query point by finding the K closest training points (using a distance metric like Euclidean) and assigning the majority class among those K neighbours. For regression, it returns the mean of K neighbours' values.

Q2. Why is KNN called a 'lazy learner'?

Answer: Because it performs no actual learning/training phase. It simply memorises the training dataset and defers all computation to prediction time. Contrast with 'eager learners' (e.g., Decision Trees) that build a generalised model during training.

Q3. How do you choose the optimal value of K?

Answer: Use cross-validation: test multiple K values (e.g., 1 to 30) on a validation set and plot accuracy vs K. Choose K at the 'elbow' where accuracy stabilises. Rule of thumb: $K \approx \sqrt{n}$. For binary classification, always use odd K to avoid tie votes.

Q4. Why is feature normalisation/scaling important for KNN?

Answer: KNN relies on distance metrics. Features with large value ranges (e.g., salary: 0–100,000) will dominate the distance calculation over features with small ranges (e.g., age: 0–100). Normalisation (Min-Max to [0,1] or Z-score standardisation) ensures all features contribute equally to the distance.

Q5. What is the curse of dimensionality in KNN?

Answer: In high-dimensional spaces, data points become increasingly equidistant from each other. The notion of 'nearest neighbour' loses statistical significance. Additionally, the volume of space grows exponentially with dimensions, requiring exponentially more data to maintain density. Solution: apply PCA or other dimensionality reduction before KNN.

Q6. What is the time complexity of KNN prediction?

Answer: Brute force: $O(n \cdot d)$ per query, where n = training samples, d = features. With KD-Tree: $O(d \cdot \log n)$ for low dimensions ($d < 20$). Space complexity: $O(n \cdot d)$ to store the entire training set.

Q7. How does KNN handle multi-class classification?

Answer: Naturally and without modification. The majority vote is taken among K neighbours across all classes. The class with the most votes wins. No one-vs-rest or one-vs-one decomposition is needed, unlike SVM.

Q8. What is weighted KNN and why is it used?

Answer: In standard KNN, all K neighbours vote equally. In weighted KNN, each neighbour's vote is inversely proportional to its distance (weight = $1/\text{distance}$). This gives closer neighbours more influence. Useful when some neighbours are much closer than others. Set weights='distance' in sklearn's KNeighborsClassifier.

Q9. How does KNN compare to SVM?

Answer: KNN: lazy learner, no training, slow prediction ($O(n \cdot d)$), very interpretable, naturally handles any boundary shape, high memory. SVM: eager learner, slow training ($O(n^2 \text{ to } n^3)$), fast prediction, less interpretable, excellent in high-dimensional spaces with kernel trick, low memory (stores only SVs). Choose KNN for small datasets; SVM for high-dimensional, medium-sized datasets.

Q10. What are the main limitations of KNN?

Answer: 1) Slow prediction for large n ($O(n \cdot d)$). 2) High memory — stores all training data. 3) Sensitive to irrelevant features and feature scale. 4) Performance degrades in high dimensions (curse of dimensionality). 5) Sensitive to imbalanced classes. 6) No compact model representation.

Q11. Can KNN be used for regression? How?

Answer: Yes. Instead of majority voting, KNN regression averages (mean) the output values of the K nearest neighbours. Use sklearn's `KNeighborsRegressor`. For weighted regression, the weighted mean is computed using $1/\text{distance}$ weights.

Q12. What distance metrics can KNN use?

Answer: Euclidean (L_2) — most common; Manhattan (L_1) — robust to outliers; Minkowski (generalisation); Hamming — for categorical data; Cosine similarity — for text/NLP; Mahalanobis — accounts for feature correlations.

21 SHORT REVISION NOTES

KNN Full Form	K-Nearest Neighbors Algorithm
Type	Supervised Learning — Classification & Regression
Learning Style	Lazy Learner / Instance-Based / Non-Parametric
Core Idea	Classify based on majority vote of K nearest training neighbours
Distance Formula	Euclidean: $d = \sqrt{\sum (p_i - q_i)^2}$
Hyperparameter	K = number of neighbours (tuned via cross-validation)
Small K effect	Overfitting — high variance, jagged boundary
Large K effect	Underfitting — high bias, over-smooth boundary
Optimal K rule	$K \approx \sqrt{n}$; use odd K for binary classification
Training Time	$O(1)$ — no training phase
Prediction Time	$O(n \cdot d)$ brute force; $O(d \cdot \log n)$ with KD-Tree
Space Complexity	$O(n \cdot d)$ — stores entire training set
Feature Scaling	MANDATORY — normalise before applying KNN
High Dimensions	Performance degrades — curse of dimensionality
sklearn Class	KNeighborsClassifier / KNeighborsRegressor
Key Parameters	n_neighbors, metric, weights, algorithm
Advantages	Simple, flexible, multi-class, no assumptions
Disadvantages	Slow prediction, high memory, feature scale sensitive

22 SUMMARY — KNN IN 5 KEY POINTS

1

DEFINITION

KNN is a supervised, non-parametric, lazy machine learning algorithm that classifies a new data point based on the majority class of its K nearest neighbours in the training dataset, measured by a distance metric (usually Euclidean).

2

CORE MECHANISM

Compute distance from query to all training points → Sort → Select K nearest → Majority vote → Predict class. No explicit training; all computation at prediction time.

3

CRITICAL HYPERPARAMETER: K

Small K = overfitting (noisy boundary). Large K = underfitting (over-smooth). Optimal K is found via cross-validation. Rule of thumb: $K \approx \sqrt{n}$. Always odd K for binary.

4

KEY STRENGTHS & WEAKNESSES

Strengths: Simple, no assumptions, natural multi-class, flexible boundaries. Weaknesses: $O(n \cdot d)$ prediction cost, high memory, sensitive to feature scale and irrelevant features, curse of dimensionality in high-d spaces.

5

REAL-WORLD IMPACT

KNN powers recommendation systems, medical diagnosis, fraud detection, image recognition, document classification, and many more applications where similarity-based reasoning is natural and effective.

This document covered: Introduction | Definition | Math Foundations | Algorithm Steps | Pseudocode | Manual Example | Python Code (Scratch + sklearn) | Flowchart | K Selection | Advantages | Disadvantages | Applications | Comparison Table | Properties | Complexity Analysis | 12 Interview Q&A | Revision Notes | Summary

Best of luck in your exams, assignments, and interviews!